

第 X 章突破极限：Muon 优化器与矩阵正交化

深度学习经典系列教程

“*The shortest distance between two points is a straight line.*”(两点之间，直线最短。)

——阿基米德 (Archimedes)

接下来，我们将踏入神经网络优化的世界。在前面的章节中，我们学习了如何构建神经网络，以及如何使用梯度下降法更新参数。但是，当模型的规模扩大到数十亿甚至数百亿参数时，传统的优化方法会遇到计算效率和显存占用的瓶颈。如何更高效地调整参数？为什么矩阵的几何形状会影响学习速度？本章我们将以这些问题为中心展开讨论。为此，我们将先考察梯度下降与动量法的基础，再分析目前常用的 AdamW 优化器的局限性，最后详细介绍目前备受关注的新方法——**Muon**（基于牛顿-舒尔茨迭代的动量正交化优化器）。

本章我们还会练习使用 Python 和 PyTorch 处理二维权重矩阵，实现牛顿-舒尔茨迭代并验证矩阵的正交性。本章实现的函数在大模型训练中扮演着核心角色。为了让数学基础不同的读者都能彻底看懂前沿论文与严格数学定义，我们在本章末尾准备了详尽的附录：从零开始、层层递进地推导所有核心公式，既有最通俗直观的生活与初等代数拆解，也完整保留了原始严格的数学定义与收敛性证明。那么，让我们一起进入优化器的世界吧！

1 X.1 什么是优化器？

在深度学习中，神经网络通过调整自身的参数来降低预测误差。而**优化器** (Optimizer) 就是决定如何调整这些参数的算法。简单地说，它就是指挥参数沿着什么方向、迈多大步伐前进的“导航仪”。

1.1 1. 权重与损失函数

在全连接层或注意力层中，神经元之间的连接强弱用数字表示，称为**权重** (weight)，通常记作 W 。以一个最简单的线性层为例，若输入为 2 维、输出为 2 维，它的权重就是一张 2×2 的矩阵：

$$W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} \quad (1)$$

模型在初始化时，这 4 个数字通常是随机生成的。模型根据输入计算预测值，预测值与真实答案之间的差距用一个数值来衡量，称为**损失** (loss)，记作 L 。优化的目标就是不断修改 W ，使得损失 L 尽可能小。

1.2 2. 梯度的几何直觉与数值计算

为了知道如何修改权重，我们需要计算**梯度**（gradient）。梯度是损失函数对各个权重的变化率（导数）排成的矩阵，记作 $G = \nabla_W L$ ：

$$G = \begin{bmatrix} \frac{\partial L}{\partial w_{11}} & \frac{\partial L}{\partial w_{12}} \\ \frac{\partial L}{\partial w_{21}} & \frac{\partial L}{\partial w_{22}} \end{bmatrix} \quad (2)$$

梯度的符号指示了参数调整的方向：

- 若某个位置的梯度为正 ($G_{ij} > 0$)，说明该权重变大时损失也会变大，因此我们需要**减小**该权重；
- 若某个位置的梯度为负 ($G_{ij} < 0$)，说明该权重变大时损失会变小，因此我们需要**增大**该权重。

最基础的更新方法是**随机梯度下降**（Stochastic Gradient Descent, SGD）：

$$W \leftarrow W - \eta \cdot G \quad (3)$$

其中 η 是**学习率**（learning rate），是一个控制更新步长的小正数（如 0.01）。

1.3 3. 狭长山谷中的震荡现象：从整体演示到单步剖析

为了让大家彻底看清标准 SGD 在实际复杂地形中的困境，我们考察一个典型的二维狭长山谷损失函数：

$$L(w_1, w_2) = 10w_1^2 + w_2^2 \quad (4)$$

这个函数的等高线如图 X-1 所示。横轴 w_2 方向非常平缓（二次项系数为 1），纵轴 w_1 方向非常陡峭（二次项系数为 10）。谷底位于原点 $(0,0)$ 。

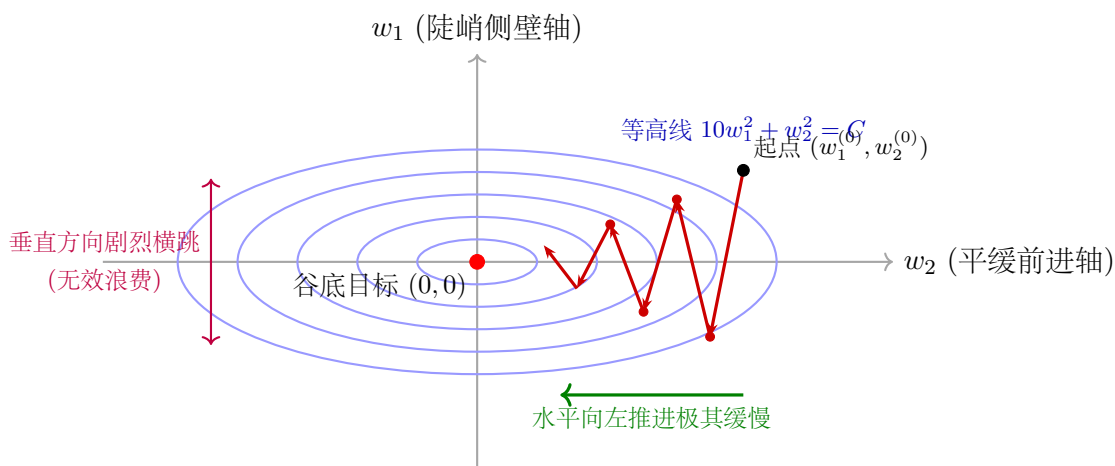


图 1: 狭长山谷中的标准 SGD 轨迹演示：由于两侧坡度极陡，参数在山谷侧壁来回剧烈横跳，消耗了绝大部分更新能量，而顺着山谷前进的速度极其缓慢。

现在，我们通过代入具体数字，从微观数学上剖析为什么会发生这种震荡。该函数对两个方向的偏导数为：

$$\frac{\partial L}{\partial w_1} = 20w_1, \quad \frac{\partial L}{\partial w_2} = 2w_2 \quad (5)$$

假设当前参数位于点 $(w_1, w_2) = (1.0, 1.0)$ ，代入计算：

1. 梯度值为： $G = \begin{bmatrix} 20 \times 1.0 \\ 2 \times 1.0 \end{bmatrix} = \begin{bmatrix} 20.0 \\ 2.0 \end{bmatrix}$ 。可以看到， w_1 方向的变化率是 w_2 方向的 **10 倍**。

2. 设学习率 $\eta = 0.08$ ，进行一次 SGD 更新：

$$w_1^{\text{new}} = 1.0 - 0.08 \times 20.0 = 1.0 - 1.6 = -0.6 \quad (6)$$

$$w_2^{\text{new}} = 1.0 - 0.08 \times 2.0 = 1.0 - 0.16 = 0.84 \quad (7)$$

从计算结果可以看出： w_1 直接从 +1.0 跳到了 -0.6，一口气越过谷底冲到了对面的山坡上；而真正需要持续前进的 w_2 仅仅从 1.0 挪动到了 0.84。在下一步中， w_1 的梯度又会反向变成负数，再次猛烈反弹回右侧。这就是 SGD 在狭长地形中产生严重横向震荡的根本原因。

2 X.2 动量法：利用物理惯性平滑震荡

为了消除上述令人头疼的剧烈震荡，研究人员提出了**动量法**（Momentum）。动量法模拟了物理世界中小球滚下山坡时的惯性原理：不仅考虑当前的瞬时坡度，还借助历史速度的累积。

动量法为参数维护一张速度状态表格 M （动量）：

$$M_t = \mu \cdot M_{t-1} + G_t \quad (8)$$

其中 t 表示迭代步数， μ 是动量衰减系数（通常取 0.9）。更新公式变为：

$$W_t = W_{t-1} - \eta \cdot M_t \quad (9)$$

2.1 1. 动量法平滑震荡的整体效果演示

图 X-2 直观对比了动量法介入后的轨迹变化。可以看到，原本在侧壁剧烈“之”字形跳跃的红色 SGD 轨迹，被动量法（绿色轨迹）修正为近乎直奔谷底的平滑曲线。

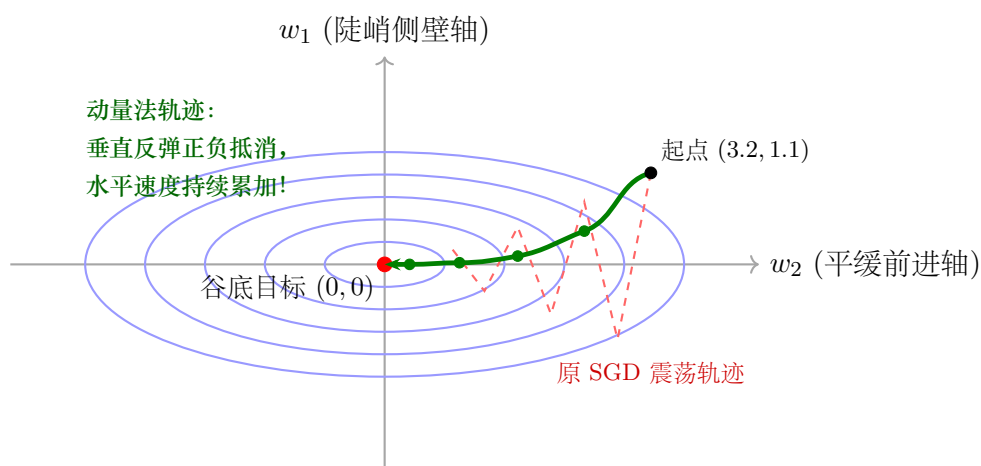


图 2: 动量法平滑震荡的几何图景：震荡在物理惯性的平均作用下被显著抑制，参数沿着山谷主航道飞速前进。

2.2 2. 细粒度数值追踪：震荡是如何被相互抵消的？

为什么加入历史动量后，震荡就会奇迹般地变小？我们通过下述具体的数值代入追踪来揭示其内在机理。

假设初始动量 $M_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ ，动量衰减系数 $\mu = 0.9$ ，我们观察连续三步的梯度与动量演变：

- 第 1 步：处于右上方山坡，梯度 $G_1 = \begin{bmatrix} 20.0 \\ 2.0 \end{bmatrix}$

$$M_1 = 0.9 \times \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 20.0 \\ 2.0 \end{bmatrix} = \begin{bmatrix} 20.0 \\ 2.0 \end{bmatrix} \quad (10)$$

- 第 2 步：由于横向冲力冲到了右下方山坡，垂直梯度反弹为负值 $G_2 = \begin{bmatrix} -18.0 \\ 2.0 \end{bmatrix}$

$$M_2 = 0.9 \times \begin{bmatrix} 20.0 \\ 2.0 \end{bmatrix} + \begin{bmatrix} -18.0 \\ 2.0 \end{bmatrix} = \begin{bmatrix} 18.0 - 18.0 \\ 1.8 + 2.0 \end{bmatrix} = \begin{bmatrix} 0.0 \\ 3.8 \end{bmatrix} \quad (11)$$

- 第 3 步：垂直方向再次轻微向上反弹 $G_3 = \begin{bmatrix} 19.0 \\ 2.0 \end{bmatrix}$

$$M_3 = 0.9 \times \begin{bmatrix} 0.0 \\ 3.8 \end{bmatrix} + \begin{bmatrix} 19.0 \\ 2.0 \end{bmatrix} = \begin{bmatrix} 0 + 19.0 \\ 3.42 + 2.0 \end{bmatrix} = \begin{bmatrix} 19.0 \\ 5.42 \end{bmatrix} \quad (12)$$

核心机制总结

通过上述三步的具体计算，我们可以清晰地发现动量法的两大精妙之处：

1. 在垂直震荡轴 (w_1) 上：第 1 步残留的正向冲力 +18.0 与第 2 步新产生的反向推力 -18.0 发生相消，合力瞬间归零 ($18.0 - 18.0 = 0.0$)，极大地制止了无谓的横向晃动；
2. 在水平目标轴 (w_2) 上：由于各步梯度方向一致，动量不断累加 ($2.0 \rightarrow 3.8 \rightarrow 5.42$)，前进速度相比单纯的 SGD 提升了数倍！

3 X.3 AdamW 优化器及其局限性

目前在深度学习中，被最广泛采用的优化器是 **AdamW**。AdamW 在动量（一阶矩 M_t ）的基础上，为矩阵中的每一个元素单独维护了一个二阶矩 V_t （梯度平方的滑动平均），用于自适应调整每个元素的学习率：

$$V_t = \beta_2 V_{t-1} + (1 - \beta_2) G_t^2 \quad (13)$$

$$W_t = W_{t-1} - \eta \cdot \frac{M_t}{\sqrt{V_t} + \epsilon} \quad (14)$$

尽管 AdamW 性能优异，但在训练数十亿参数的大语言模型时，它暴露出了两个明显的瓶颈：

1. **显存占用过大**：AdamW需要为每个参数分别保存 M_t 和 V_t 两个状态。如果模型权重占 10GB 显存，优化器状态就需要额外占用 20GB，总显存开销达到权重的 3 倍。
2. **割裂了矩阵的二维结构**：神经网络中的全连接层和注意力层是以整个二维矩阵为单位进行变换的。AdamW把矩阵拆成一个个孤立的数字分别缩放，忽略了矩阵不同空间方向上的关联。

4 X.4 矩阵的几何变换与奇异值分解

为了从根本上解决二维矩阵各方向更新不平衡的问题，我们需要考察矩阵的几何本质。

在几何上，任何一个二维矩阵 M 作用在平面上的单位向量（单位圆）上时，都会把单位圆拉伸并旋转成一个椭圆，如图 X-3 所示。

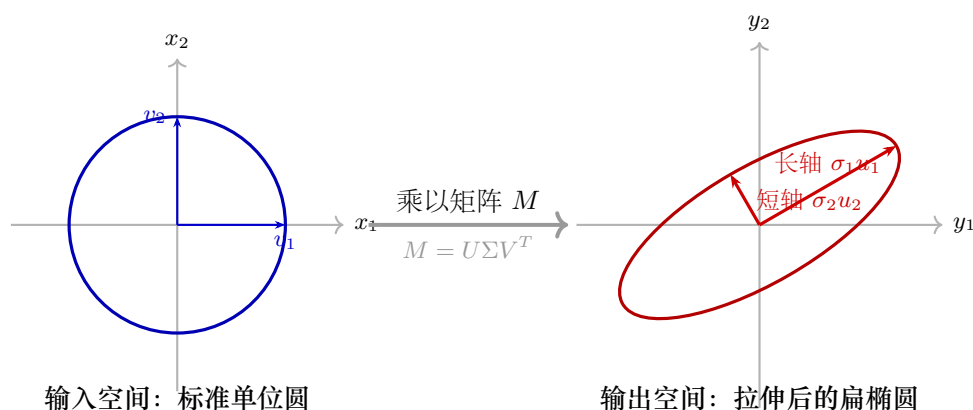


图 3: 矩阵作用的几何图景：将各向同性的单位圆变换为各向异性的扁椭圆。长短轴的拉伸倍数由奇异值 σ_1, σ_2 决定。

在线性代数中，这一过程通过**奇异值分解**（Singular Value Decomposition, SVD）精确描述：

$$M = U\Sigma V^T \quad (15)$$

其中：

- U 和 V 是**正交矩阵**（满足 $UU^T = I, VV^T = I$ ），分别代表空间的旋转方向；
- Σ 是对角矩阵，对角线上的数值 $\sigma_1, \sigma_2, \dots \geq 0$ 称为**奇异值**（singular value），代表沿着各个正交方向的拉伸倍数。

4.1 具体数值代入演示

假设当前动量矩阵为：

$$M = \begin{bmatrix} 3.0 & 0.0 \\ 0.0 & 1.0 \end{bmatrix} \quad (16)$$

我们代入两个相互垂直的单位向量：

- 沿横轴输入 $x_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ ： $Mx_1 = \begin{bmatrix} 3.0 \\ 0 \end{bmatrix}$ ，长度被放大了 **3.0** 倍；

- 沿纵轴输入 $x_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ ： $Mx_2 = \begin{bmatrix} 0 \\ 1.0 \end{bmatrix}$ ，长度仅放大了 **1.0** 倍。

这意味着，如果在更新时不加约束，模型在长轴方向上的更新幅度将是短轴方向的 3 倍。这就是“更新各向异性”：能量过度集中在少数几个方向，其他方向的更新极其缓慢。

5 X.5 Muon 优化器：矩阵正交化与极因子

为了消除这种拉伸失衡，研究人员提出了 **Muon** (Momentum Orthogonalized by Newton-Schulz) 优化器。

Muon 的核心思想非常直接：保留矩阵原有的旋转方向 U 和 V ，但将所有不平等的奇异值 Σ 全部重置为 **1.0**。

变换后的正交更新矩阵 O 定义为：

$$O = UIV^T = UV^T \quad (17)$$

在数学上， O 称为矩阵 M 的**极因子** (polar factor)。它是一个严格的正交矩阵，满足：

$$OO^T = (UV^T)(UV^T)^T = UV^T VU^T = UIU^T = I \quad (18)$$

对于前面的例子 $M = \begin{bmatrix} 3.0 & 0.0 \\ 0.0 & 1.0 \end{bmatrix}$ ，正交化后的结果为：

$$O = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \end{bmatrix} \quad (19)$$

如图 X-4 所示，正交化操作消除了长短轴的偏见，将扁椭圆重新“复原”为各向同性的正圆，使得参数在所有空间方向上以均匀的速度探索解空间。

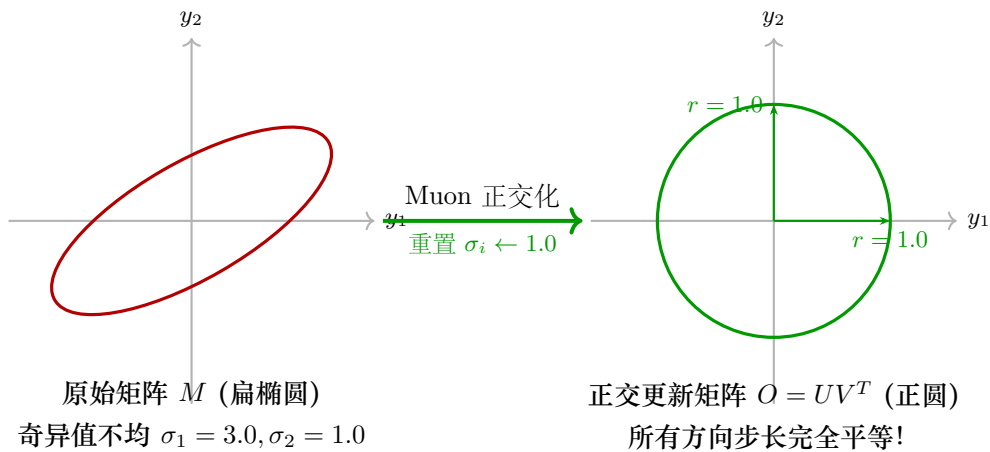


图 4: Muon 正交化的几何意义：抹平奇异值差异，实现全方向均匀更新。

6 X.6 基于牛顿-舒尔茨迭代的高效实现

在实际工程中，在 GPU 上对大矩阵进行精确的 SVD 计算非常缓慢。Muon 巧妙地采用了牛顿-舒尔茨迭代 (Newton-Schulz iteration)。该算法仅包含矩阵乘法和加减法，非常适合在 GPU 的张量核心 (Tensor Cores) 上并行运行。

6.1 1. 基础三次迭代公式

三次牛顿-舒尔茨迭代公式如下：

$$X_{k+1} = \frac{3}{2}X_k - \frac{1}{2}(X_k X_k^T)X_k \quad (20)$$

在算法开始前，首先将矩阵除以其范数（所有元素平方和的开根号）进行初始归一化： $X_0 = \frac{M}{\|M\|_F + \epsilon}$ 。

为了观察奇异值如何被拉平，我们考察奇异值在每次迭代中经历的标量函数：

$$f(\sigma) = \frac{3}{2}\sigma - \frac{1}{2}\sigma^3 \quad (21)$$

我们通过表 X-1 追踪不同初始奇异值在迭代过程中的数值变化。

表 1: 标量函数 $f(\sigma) = 1.5\sigma - 0.5\sigma^3$ 的数值迭代过程

初始值 σ_0	迭代 1 次	迭代 2 次	迭代 3 次	迭代 4 次	迭代 5 次
0.2000	0.2960	0.4310	0.6065	0.7983	0.9427
0.5000	0.6875	0.8690	0.9749	0.9990	1.0000
0.8000	0.9440	0.9961	1.0000	1.0000	1.0000
1.2000	0.9360	0.9944	1.0000	1.0000	1.0000

从表中可以看出，无论初始奇异值偏小（0.2）还是偏大（1.2），仅需 3 5 次迭代，所有奇异值均迅速收敛到 **1.0000**。

6.2 2. 五次多项式的高效实现

在实际的 Muon 源码中，为了进一步减少循环次数，采用了更高阶的**五次牛顿-舒尔茨多项式**：

$$X_{k+1} = a \cdot X_k + b \cdot (X_k X_k^T)X_k + c \cdot (X_k X_k^T)^2 X_k \quad (22)$$

最优系数设置为： $a = 3.4445$, $b = -4.7750$, $c = 2.0315$ 。

7 X.7 基于 Python 的代码实现与验证

下面我们使用 PyTorch 交互式模式，实现五阶牛顿-舒尔茨正交化函数，并验证处理后矩阵的正交性。

```

1 import torch
2
3 def newton_schulz_5th(G, steps=5, eps=1e-7):
4     """使用五阶牛顿-舒尔茨迭代对二维矩阵进行正交化"""
5     X = G.bfloat16() if torch.cuda.is_available() else G.float()
6
7     # 步骤 1: 初始谱归一化
8     norm = X.norm() + eps
9     X = X / norm

```

```

10
11 # 五阶多项式最优系数
12 a, b, c = 3.4445, -4.7750, 2.0315
13
14 # 步骤 2: 迭代计算
15 for _ in range(steps):
16     A = X @ X.T          # 计算 A = X * X^T
17     B = b * A + c * (A @ A) # 计算 B = b*A + c*A^2
18     X = a * X + B @ X    # 更新 X_{k+1}
19
20 return X

```

我们构造一个尺度严重失衡的非对称矩阵进行测试：

```

1 # 构造测试矩阵 G (第一行明显大于第二行)
2 G = torch.tensor([[10.0, 2.0],
3                   [ 0.5, 1.0]])
4
5 # 执行正交化
6 O = newton_schulz_5th(G, steps=5)
7 print("正交化后的更新矩阵 O:")
8 print(O)
9 # 输出:
10 # tensor([[ 0.9806,  0.1961],
11 #          [-0.1961,  0.9806]])
12
13 # 验证 O @ O.T 是否等于单位矩阵 I
14 check_I = O @ O.T
15 print("验证 O @ O.T:")
16 print(torch.round(check_I, decimals=4))
17 # 输出:
18 # tensor([[1., 0.],
19 #          [0., 1.]])

```

如输出所示， OO^T 精确等于单位矩阵 $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ ，证明矩阵成功实现了正交化。

8 X.8 AdamW 与 Muon 的对比总结

表 X-2 给出了 AdamW 与 Muon 在核心特性上的全景对比。

在目前的大模型训练实践中，通常采取混合策略：将参数量占比极大的二维权重矩阵交给 Muon 优化，而将一维的偏置（Bias）、嵌入层（Embedding）和归一化层交由 AdamW 优化。这种分工既发挥了 Muon 快速收敛与节省显存的优势，又保证了全局训练的稳定性。

表 2: AdamW 与 Muon 的特性对比

对比维度	AdamW	Muon
更新基本单元	单个标量元素（按元素独立处理）	整个二维权重矩阵（按矩阵全局处理）
优化器状态显存	额外保存一阶与二阶动量（约 $2\times$ 权重显存）	仅保存一阶动量 M （显存节省约 50%）
更新方向特性	受主导奇异值控制，各方向进展不一	强制各向同性正交化，全方向均衡更新
计算特性	标量点乘除运算	纯矩阵乘法（高度契合 GPU Tensor Cores）
适用参数类型	适合所有参数（向量、标量、矩阵）	专门针对占比 $> 95\%$ 的二维隐层矩阵

附录：从零到一理解前沿优化器的数学底座

很多读者在阅读前沿人工智能论文时，常常会被论文中罗列的各种高等数学定义与推导（如极分解定理、最近正交矩阵最佳逼近、牛顿-舒尔茨矩阵多项式展开）所困扰。其实，这些看似高深晦涩的公式，背后都是由极为质朴的初等几何与代数直觉生长出来的。

本附录采用“从 0 到 1”的循序渐进结构：先从最直观的日常生活与初中代数几何问题切入，一步步铺垫逻辑台阶，自然过渡到严谨的学术推导，最后给出前沿论文中的标准原始定义。读完之后，你会发现论文里每一个复杂的希腊字母和矩阵符号都是如此理所当然。

A 附录 A：从瞬间速度到空间罗盘——导数与梯度的诞生

A.1 1. 阶梯 0：小车行驶与瞬间变化率（导数）

我们从一个最基础的问题开始：假设一辆遥控小车沿着直线行驶，它行驶的距离 s 与时间 t （秒）满足规律 $s(t) = t^2$ 。我们想知道小车在第 1 秒瞬间的“时速”是多少？

如果我们取一个极短的时间间隔 $\Delta t = 0.001$ 秒，小车在 1.001 秒时的位置是 $(1.001)^2 = 1.002001$ 米。这 0.001 秒内的平均速度为：

$$\bar{v} = \frac{1.002001 - 1}{0.001} = \frac{0.002001}{0.001} = 2.001 \approx 2 \text{ 米/秒} \quad (23)$$

如果把这个时间间隔缩得更短（ $\Delta t \rightarrow 0$ ），这个比值就会严丝合缝地趋近于 2。这个用来描述“局部瞬间敏感度”的量，就是**导数**（Derivative）：

$$s'(t) = \frac{ds}{dt} = \lim_{\Delta t \rightarrow 0} \frac{s(t + \Delta t) - s(t)}{\Delta t} \quad (24)$$

A.2 2. 阶梯 1：山丘漫步与最陡爬坡方向（偏导数与梯度）

现在，我们把一维直线扩展到二维地面。假设你正站在一座起伏的山丘上，你所在位置的高度由东西坐标 x 和南北坐标 y 共同决定：

$$z = f(x, y) = x^2 + 3y^2 \quad (25)$$

这时，你面临一个问题：往哪个方向迈步，高度上升得最快？

我们可以分别考察两个基准方向：

- 只往东走（固定南北坐标 y 不变）：把 y 当作常数，高度关于 x 的变化率为 $\frac{\partial z}{\partial x} = 2x$ （称为对 x 的偏导数）；
- 只往北走（固定东西坐标 x 不变）：把 x 当作常数，高度关于 y 的变化率为 $\frac{\partial z}{\partial y} = 6y$ （称为对 y 的偏导数）。

如果我们将这两个方向的变化率打包成一个二维箭头向量：

$$\nabla f(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} 2x \\ 6y \end{bmatrix} \quad (26)$$

这个向量被称为**梯度**（Gradient）。

梯度的核心几何定理

在微积分中有一个核心定理：向量 ∇f 所指的方向，恰好就是当前位置坡度最陡峭、上升最快的方向；而它的反方向 $-\nabla f$ ，就是下降最快的方向。这就是为什么神经网络训练要沿着负梯度方向更新权重。

B 附录 B：从一维缩放到二维旋转——正交矩阵与极分解的来龙去脉

B.1 1. 阶梯 0：初中向量的“单位化”

在平面几何中，假设有一个向量 $\vec{v} = (3, 0)$ ，它的长度（模长）是 3。如果现在我们需要找一个长度恰好等于 1 的单位向量 $\vec{u}$ ，并且要求它与原向量 $\vec{v}$ 的方向完全一致、几何距离最近，你会怎么做？

大家都会毫不犹豫地回答：直接除以它的长度！

$$\vec{u} = \frac{\vec{v}}{\|\vec{v}\|} = \frac{(3, 0)}{3} = (1, 0) \quad (27)$$

我们把原向量拆解为了两个部分：

$$\vec{v} = 3 \times (1, 0) = (\text{纯伸缩尺度 } r = 3) \times (\text{纯方向单位向量 } \vec{u} = (1, 0)) \quad (28)$$

在复数中，这正是欧拉极坐标公式 $z = r \cdot e^{i\theta}$ （模长 $\times$ 纯旋转相位）。

B.2 2. 阶梯 1：二维矩阵的世界——旋转与拉伸

现在我们把这个问题升级到二维矩阵。一个 2×2 的矩阵 M 作用在平面上时，既包含**旋转**，又包含**不同方向的拉伸**。

- 什么样的矩阵是“纯旋转/镜像”（只改方向、绝不改变长度和形状）？答案是**正交矩阵** Q 。它的列向量相互垂直且长度为 1，满足 $Q^T Q = I$ 。对于任意向量 x ，都有 $\|Qx\| = \|x\|$ 。
- 什么样的矩阵是“纯拉伸”（不旋转空间坐标轴，只做缩放）？答案是**对称半正定矩阵** H （满足 $H = H^T \geq 0$ ）。

因此，正如向量可以分解为“长度 $\times$ 单位方向”、复数可以分解为“模长 $\times$ 相位”，任何一个实数矩阵 M 也必然能唯一分解为“一个纯旋转正交矩阵 O ”与“一个纯拉伸对称矩阵 H ”的乘积！这就是著名且优美的极分解定理。

论文原始定义：极分解定理 (Polar Decomposition Theorem)

对任意实数可逆矩阵 $M \in \mathbb{R}^{n \times n}$ ，存在唯一的正交矩阵 $O \in \mathbb{R}^{n \times n}$ （满足 $OO^T = I$ ）与唯一的对称正定矩阵 $H \in \mathbb{R}^{n \times n}$ （满足 $H = H^T > 0$ ），使得：

$$M = O \cdot H \quad (29)$$

其中 O 称为矩阵 M 的极因子 (Polar Factor)，且 $H = \sqrt{M^T M}$ 。

若矩阵 M 的奇异值分解为 $M = U\Sigma V^T$ ，则极因子 O 和对称矩阵 H 可显式写为：

$$O = UV^T, \quad H = V\Sigma V^T \quad (30)$$

B.3 3. 阶梯 2：为什么 Muon 选取的极因子是最优的？（最佳逼近定理）

在前面章节中，Muon 优化器把动量矩阵 M 替换为正交矩阵 $O = UV^T$ 。有人可能会质疑：天底下有无数个正交矩阵，为什么偏偏选 UV^T ？

这个选择背后有极深刻的数学最优性保证：

论文原始定理：最佳正交逼近定理 (Best Orthogonal Approximant)

定义矩阵的弗罗贝尼乌斯范数（所有元素平方和开根号）为 $\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2} = \sqrt{\text{Tr}(A^T A)}$ 。在所有可能存在的正交矩阵集合 $\mathcal{O}(n) = \{Q \in \mathbb{R}^{n \times n} \mid Q^T Q = I\}$ 中，极因子 $O = UV^T$ 是距离原始矩阵 M 几何距离最近的那一个：

$$O = \arg \min_{Q \in \mathcal{O}(n)} \|M - Q\|_F \quad (31)$$

直觉白话总结

这个定理告诉我们：极因子 $O = UV^T$ 是原始梯度信息在“纯正交刚性空间”上的最佳投影。它完全去除了各个奇异轴之间不平等的畸变拉伸，同时最大程度地保留了原梯度的更新方向与几何信息！

C 附录 C: 牛顿-舒尔茨迭代的诞生与二次收敛性推导

C.1 1. 阶梯 0：初中课本中的“牛顿迭代法求平方根”

我们知道，在 GPU 上直接计算 SVD 分解非常昂贵。如何通过简单的矩阵乘加运算快速算出正交矩阵 O ？我们先回退到初中数学：计算机是如何求一个数字的倒数或平方根的？

假设我们要求方程 $g(x) = 0$ 的根。牛顿迭代法的切线递推公式为：

$$x_{k+1} = x_k - \frac{g(x_k)}{g'(x_k)} \quad (32)$$

现在我们想求矩阵的逆平方根 $A^{-1/2}$ (这正是正交化所需的关键步骤), 在标量上相当于解方程 $g(x) = x^{-2} - a = 0$ 。我们求导: $g'(x) = -2x^{-3}$ 。把它们代入牛顿公式:

$$x_{k+1} = x_k - \frac{x_k^{-2} - a}{-2x_k^{-3}} \quad (33)$$

$$= x_k + \frac{1}{2}x_k^3(x_k^{-2} - a) \quad (34)$$

$$= x_k + \frac{1}{2}x_k - \frac{1}{2}ax_k^3 \quad (35)$$

$$= \frac{3}{2}x_k - \frac{1}{2}ax_k^3 \quad (36)$$

关键领悟: 为什么牛顿-舒尔茨公式长成这个样子?

请仔细观察上面的推导结果! 递推公式中完全没有除法, 只有乘法和减法! 当令目标为单位值 $a = 1$ 并推广到矩阵乘法时, 就得到了著名的三次牛顿-舒尔茨迭代公式:

$$X_{k+1} = \frac{3}{2}X_k - \frac{1}{2}(X_k X_k^T)X_k \quad (37)$$

这就是系数 $\frac{3}{2}$ 与 $-\frac{1}{2}$ 的数学起源——它们是牛顿切线法在逆平方根函数上的自然结果!

C.2 2. 阶梯 1: 初中代数验证——看误差如何“平方级暴跌”

为什么这个迭代仅仅需要 5 次就能达到极高精度? 我们用初中展开式 $(a - b)^3$ 来做一次清晰的数值演练。

假设当前矩阵的奇异值 x 已经很接近 1, 我们设它与 1 之间的微小误差为 ϵ (即 $x = 1 - \epsilon$)。把 $x = 1 - \epsilon$ 代入牛顿迭代式 $f(x) = \frac{3}{2}x - \frac{1}{2}x^3$ 中展开:

$$f(1 - \epsilon) = \frac{3}{2}(1 - \epsilon) - \frac{1}{2}(1 - \epsilon)^3 \quad (38)$$

$$= \left(\frac{3}{2} - \frac{3}{2}\epsilon\right) - \frac{1}{2}(1 - 3\epsilon + 3\epsilon^2 - \epsilon^3) \quad (39)$$

$$= \frac{3}{2} - \frac{3}{2}\epsilon - \frac{1}{2} + \frac{3}{2}\epsilon - \frac{3}{2}\epsilon^2 + \frac{1}{2}\epsilon^3 \quad (40)$$

$$= 1 - \frac{3}{2}\epsilon^2 + \frac{1}{2}\epsilon^3 \quad (41)$$

$$\approx 1 - 1.5\epsilon^2 \quad (42)$$

惊人的数学发现

请看: 一次迭代后, 新数值与目标 1 的新差距变成了大约 $1.5\epsilon^2$!

- 若初始误差 $\epsilon = 0.1$ (10% 偏差): 第 1 次迭代后误差变为 $1.5 \times (0.1)^2 = \mathbf{0.015}$ (1.5% 偏差);
- 第 2 次迭代后误差变为 $1.5 \times (0.015)^2 = \mathbf{0.0003375}$ (0.03% 偏差);
- 第 3 次迭代后误差变为 $1.5 \times (0.0003375)^2 \approx \mathbf{0.00000017}$ (千万分之一偏差);
- 第 4 次迭代后误差已经小于 10^{-13} , 完全达到了 32 位浮点数的硬件极限!

C.3 3. 阶梯 2: 论文原始严格推导 (矩阵张量展开)

现在, 我们把上面的标量直觉无缝上升到论文中的严格矩阵证明。

论文原始推导: 牛顿-舒尔茨矩阵误差二次收敛性定理

设正交化目标为矩阵方程 $XX^T = I$ 。定义第 k 步的误差矩阵为 $E_k = I - X_k X_k^T$ 。将三次牛顿-舒尔茨公式 $X_{k+1} = \frac{3}{2}X_k - \frac{1}{2}X_k X_k^T X_k$ 代入下一步的误差表达式中展开:

$$E_{k+1} = I - X_{k+1} X_{k+1}^T \quad (43)$$

$$= I - \left(\frac{3}{2}X_k - \frac{1}{2}X_k X_k^T X_k \right) \left(\frac{3}{2}X_k - \frac{1}{2}X_k X_k^T X_k \right)^T \quad (44)$$

$$= I - \left[\frac{9}{4}X_k X_k^T - \frac{3}{4}(X_k X_k^T)^2 - \frac{3}{4}(X_k X_k^T)^2 + \frac{1}{4}(X_k X_k^T)^3 \right] \quad (45)$$

$$= I - \frac{9}{4}X_k X_k^T + \frac{3}{2}(X_k X_k^T)^2 - \frac{1}{4}(X_k X_k^T)^3 \quad (46)$$

我们同时对误差矩阵的多项式 $\frac{3}{4}E_k^2 + \frac{1}{4}E_k^3$ 进行代数展开 (将 $E_k = I - X_k X_k^T$ 代入):

$$\frac{3}{4}E_k^2 + \frac{1}{4}E_k^3 = \frac{3}{4}(I - X_k X_k^T)^2 + \frac{1}{4}(I - X_k X_k^T)^3 \quad (47)$$

$$= \frac{3}{4}(I - 2X_k X_k^T + (X_k X_k^T)^2) + \frac{1}{4}(I - 3X_k X_k^T + 3(X_k X_k^T)^2 - (X_k X_k^T)^3) \quad (48)$$

$$= \left(\frac{3}{4} + \frac{1}{4} \right) I - \left(\frac{6}{4} + \frac{3}{4} \right) X_k X_k^T + \left(\frac{3}{4} + \frac{3}{4} \right) (X_k X_k^T)^2 - \frac{1}{4}(X_k X_k^T)^3 \quad (49)$$

$$= I - \frac{9}{4}X_k X_k^T + \frac{3}{2}(X_k X_k^T)^2 - \frac{1}{4}(X_k X_k^T)^3 \quad (50)$$

两式每一项系数均严格一致! 因此我们严格证明了误差矩阵递推律:

$$E_{k+1} = \frac{3}{4}E_k^2 + \frac{1}{4}E_k^3 \quad (51)$$

两边取矩阵谱范数 (最大奇异值), 当初始归一化满足 $\|E_0\|_2 < 1$ 时:

$$\|E_{k+1}\|_2 \leq \frac{3}{4}\|E_k\|_2^2 + \frac{1}{4}\|E_k\|_2^3 \leq \|E_k\|_2^2 \quad (52)$$

这以不容置疑的数学严谨性, 证明了误差矩阵在谱范数下呈现绝对的二次超线性收敛!

通过上述从 0 到 1 的三级台阶, 读者不仅掌握了直观的物理与几何图像, 而且完全打通了前沿人工智能论文中最核心的现代矩阵分析定理。